

מבחן תרגול - אפיון ותכן מערכות

נושאים: ניתוח דרישות, UML Diagrams, ארכיטקטורה ומתודולוגיות פיתוח

משך הבחינה: 90 דקות

ניקוד כולל: 100 נקודות

חומר עזר: לפי הנחיית המרצה

שם:

תאריך:

הנחיות כלליות

- יש לענות על כל השאלות. בשאלות אמריקאיות יש לבחור תשובה אחת בלבד, אלא אם צוין אחרת.
- בשאלות פתוחות יש לנמק בקצרה ובאופן ענייני. תשובה ללא נימוק עלולה לקבל ניקוד חלקי בלבד.
- בשאלות UML יש להתייחס לסמלים, לקשרים, למריבויות, לאחריות המחלקות ולזרימת התהליך.
- קטעי הקוד/פסאודו-קוד מיועדים להבנת תכנון המערכת ולא להרצה בפועל.

מבנה הבחינה

חלק	סוג שאלות	ניקוד
א	שאלות אמריקאיות	32 נקודות
ב	נכון / לא נכון	18 נקודות
ג	שאלות ויזואליות, UML ותרשימים	30 נקודות
ד	שאלות פתוחות ותכנון	20 נקודות

טיפ: במבחן מסוג זה לא מספיק לזהות מושג. חשוב להסביר למה בחרת בתרשים, בקשר UML או במתודולוגיה מסוימת.

חלק א - שאלות אמריקאיות (32 נקודות)

בחרו את התשובה הנכונה ביותר. כל שאלה שווה 4 נקודות.

1. איזו מהאפשרויות הבאות היא דוגמה לדרישה פונקציונלית?

 - a. המערכת תגיב לכל פעולה תוך פחות משתי שניות.
 - b. המערכת תאפשר למשתמש לאפס סיסמה באמצעות אימייל.
 - c. המערכת תהיה זמינה 99.9% מהזמן.
 - d. מסך הבית יהיה נגיש גם למשתמשים עם לקות ראייה.
2. בתרשים Use Case, מה בדרך כלל מייצג שחקן (Actor)?

 - a. טבלה במסד הנתונים.
 - b. ישות חיצונית שמתקשרת עם המערכת.
 - c. מחלקה פנימית שמבצעת חישוב.
 - d. רכיב UI בלבד.
3. קשר הכללה (Generalization) בתרשים מחלקות מתאים במיוחד כאשר:

 - a. מחלקה אחת משתמשת זמנית במחלקה אחרת.
 - b. יש יחס של "סוג של" בין מחלקת אב למחלקת בן.
 - c. יש קשר בין מסד נתונים לשרת.
 - d. אובייקט אחד שולח הודעה בזמן ריצה.
4. מה המטרה המרכזית של תרשים Sequence?

 - a. להציג את מבנה הטבלאות במסד הנתונים.
 - b. להציג את סדר ההודעות בין אובייקטים לאורך זמן.
 - c. להציג את היררכיית המחלקות בלבד.
 - d. להגדיר את מראה המסכים במערכת.
5. בארכיטקטורת שכבות, מהו יתרון מרכזי?

 - a. כל הקוד נמצא בקובץ אחד ולכן קל יותר לערוך אותו.
 - b. ניתן להפריד אחריות בין ממשק משתמש, לוגיקה עסקית וגישה לנתונים.
 - c. אין צורך בבדיקות מערכת.
 - d. כל משתמש יכול לגשת ישירות למסד הנתונים.
6. באיזו מתודולוגיה מתאים במיוחד לעבוד כאשר הדרישות משתנות לעיתים קרובות?

 - a. Waterfall בלבד.
 - b. Agile / Scrum.
 - c. Big Bang ללא תכנון.
 - d. תיעוד בלבד ללא פיתוח.

7. מהי מטרת Acceptance Criteria בדרישת משתמש?

- a. לתאר איך המערכת נראית מבחינה גרפית בלבד.
- b. להגדיר תנאים ברורים שלפיהם יודעים שהדרישה בוצעה בהצלחה.
- c. להחליף את הצורך בבדיקות.
- d. להגדיר את שמות המחלקות בקוד.

8. איזה תרשים UML מתאים ביותר להצגת מצבי אובייקט והמעברים ביניהם?

- a. State Machine Diagram.
- b. Deployment Diagram.
- c. Use Case Diagram.
- d. Package Diagram.

חלק ב - נכון / לא נכון (18 נקודות)

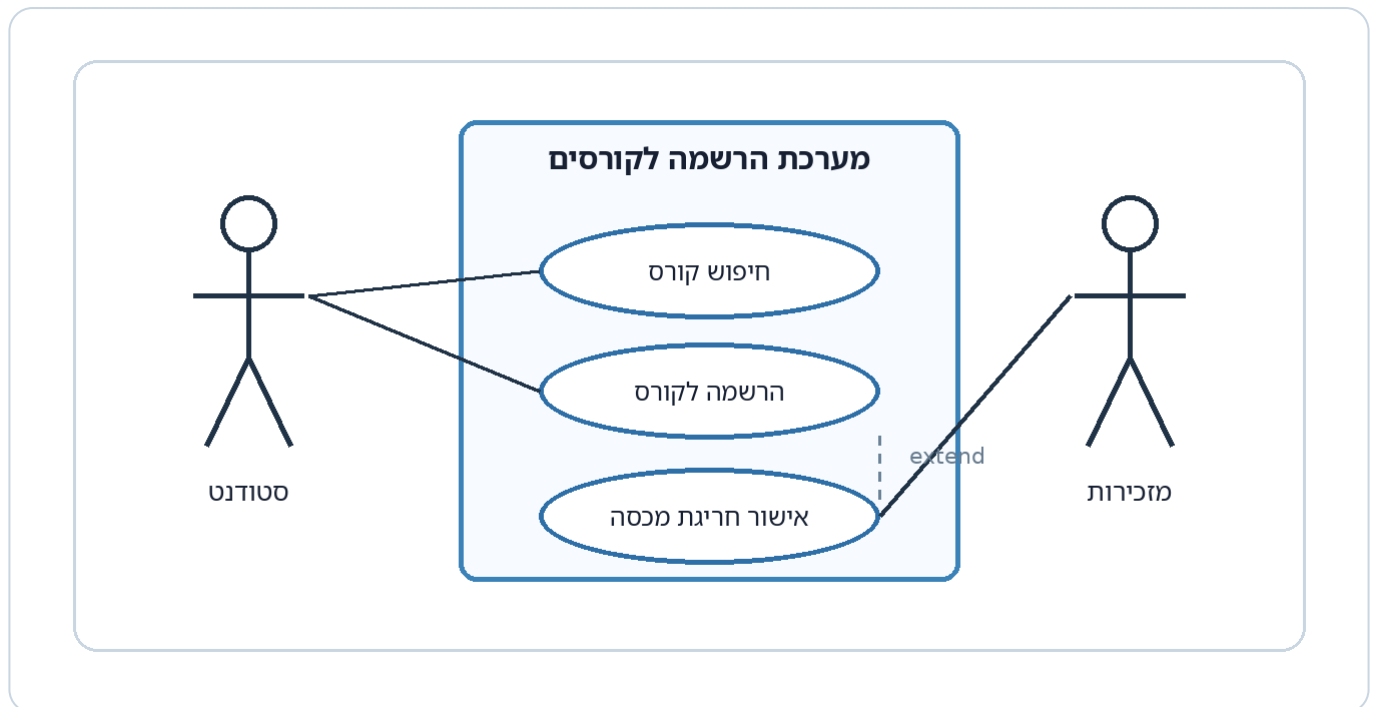
סמנו נכון או לא נכון. כל סעיף שווה 2 נקודות.

#	טענה	נכון	לא נכון
1	דרישה לא פונקציונלית מתארת לרוב איכות או מגבלה של המערכת, כמו ביצועים, אבטחה או זמינות.	☐	☐
2	תרשים Use Case נועד בעיקר להציג את קוד המחלקות ואת המימוש הפנימי.	☐	☐
3	Multiplicity בתרשים מחלקות עוזרת להבין כמה מופעים של מחלקה אחת יכולים להיות קשורים למופע של מחלקה אחרת.	☐	☐
4	ב-Agile מקובל לקבל משוב מהלקוח לאורך הדרך ולא רק בסוף הפרויקט.	☐	☐
5	בארכיטקטורת MVC, ה-Model אחראי בדרך כלל על הנתונים והלוגיקה העסקית.	☐	☐
6	תרשים Activity מתאים לתיאור זרימת פעולות ותנאים בתהליך עסקי.	☐	☐
7	במתודולוגיית Waterfall אין צורך באפיון דרישות לפני הפיתוח.	☐	☐
8	תרשים Deployment יכול להציג על אילו שרתים או רכיבי חומרה המערכת נפרסת.	☐	☐
9	דרישה עמומה כמו "המערכת תהיה נוחה" נחשבת דרישה טובה ומדידה כפי שהיא.	☐	☐

חלק ג - שאלות ויזואליות, UML ותרשימים (30 נקודות)

שאלה 1 - ניתוח תרשים Use Case (8 נקודות)

לפניכם תרשים Use Case חלקי עבור מערכת לניהול הרשמה לקורסים במכללה.



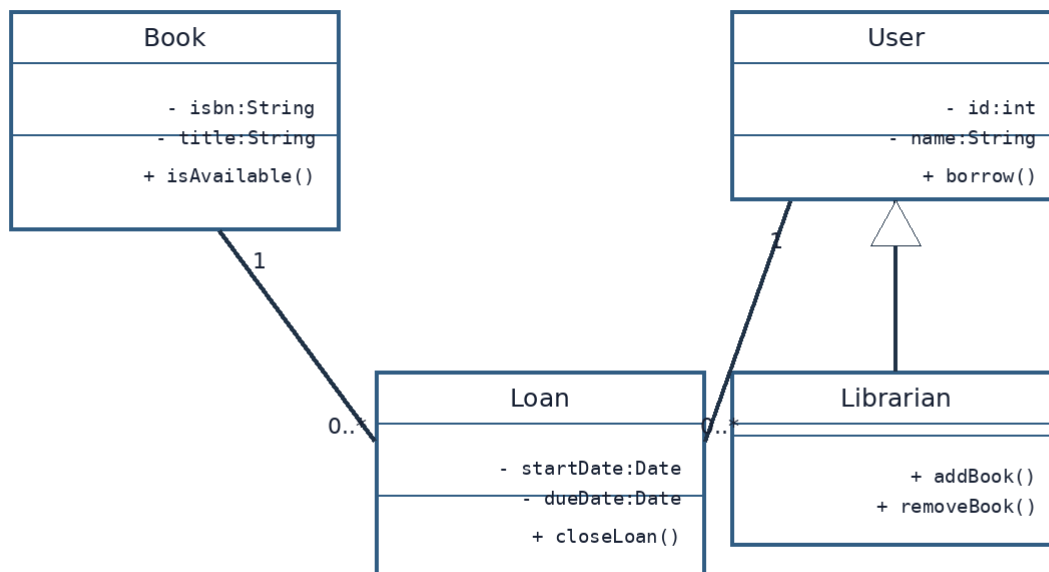
1. זהו שני שחקנים ושני מקרי שימוש מרכזיים בתרשים.

2. הסבירו האם הקשר בין "הרשמה לקורס" לבין "אישור חריגת מכסה" מתאים יותר ל-include או ל-extend. נמקו.

3. הציעו Use Case נוסף שחסר לדעתכם בתרשים.

שאלה 2 - תרשים מחלקות (8 נקודות)

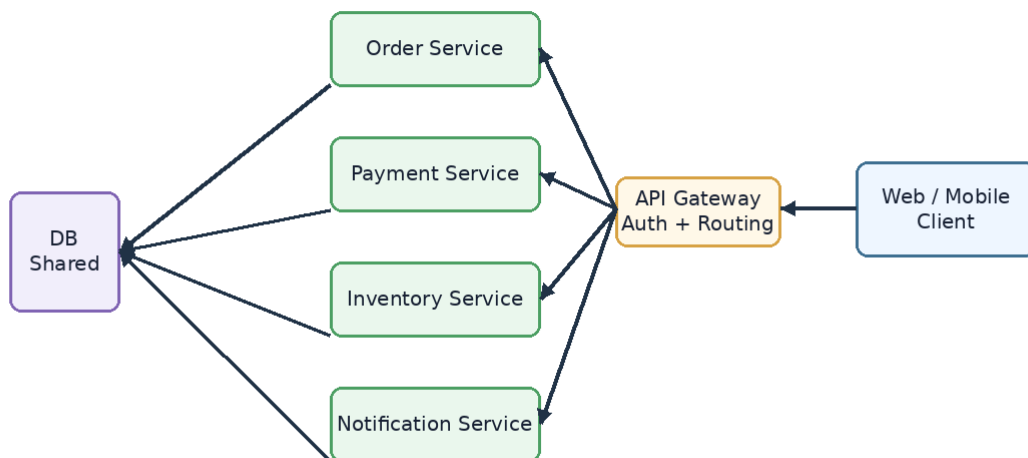
לפניכם תרשים מחלקות חלקי עבור מערכת ספרייה.



1. הסבירו את משמעות הקשר בין User לבין Librarian.
2. לפי המריבויות בתרשים, האם משתמש יכול להיות קשור למספר השאלות? נמקו.
3. הציעו שיפור אחד לתרשים כדי לייצג מצב שבו ספר יכול להיות "שמור" למשתמש, אך עדיין לא מושאל.

שאלה 3 - תרשים ארכיטקטורה (7 נקודות)

לפניכם תרשים ארכיטקטורה כללי עבור מערכת הזמנות אונליין.



1. ציינו יתרון אחד וחיסרון אחד של שימוש ב-API Gateway בתרשים.
2. מה עלול להיות בעייתי בכך שכל השירותים משתמשים באותו DB משותף?
3. הציעו שינוי ארכיטקטוני אחד שישפר עצמאות או סקיילביליות.

שאלה 4 - קוד/פסאודו-קוד ותכן (7 נקודות)

במערכת קיימת פונקציה אחת גדולה שמטפלת גם בולידציה, גם בחישוב מחיר, גם בשמירה למסד נתונים וגם בשליחת הודעה ללקוח.

```

function createOrder(request) {
  validateUser(request.user);
  calculateDiscount(request.items);
  saveOrderToDatabase(request);
  sendSmsToCustomer(request.phone);
}
  
```

1. איזה עיקרון תכן נפגע כאן? הסבירו בקצרה.
2. הציעו חלוקה לשתי מחלקות/שירותים לפחות.
3. איזה תרשים UML יכול לעזור להציג את זרימת הפעולות בתהליך הזה?

חלק ד - שאלות פתוחות ותכנון (20 נקודות)

שאלה 1 - ניתוח דרישות (6 נקודות)

לקוח מבקש מערכת להזמנת תורים למרפאה. הוא אומר: "אני רוצה שהמערכת תהיה מהירה, פשוטה, ותאפשר למטופלים לקבוע תור בלי להתקשר".

- 1. נסחו שתי דרישות פונקציונליות מדידות וברורות.
- 2. נסחו שתי דרישות לא פונקציונליות מדידות וברורות.
- 3. כתבו Acceptance Criterion אחד עבור קביעת תור.

שאלה 2 - בחירת תרשים UML מתאים (5 נקודות)

עבור כל צורך, בחרו את תרשים ה-UML המתאים ביותר והסבירו במשפט אחד:

צורך	תרשים מתאים	נימוק קצר
להראות אילו משתמשים מבצעים אילו פעולות במערכת		
להראות את סדר ההודעות בין אובייקטים בזמן ביצוע הזמנה		
להראות קשרי ירושה, מאפיינים ומתודות בין מחלקות		

שאלה 3 - מתודולוגיות פיתוח (5 נקודות)

חברה מפתחת מערכת חדשה, אך הלקוח עדיין לא יודע בדיוק אילו פיצ'רים יהיו בגרסה הסופית. הצוות קטן, ויש צורך בהדגמות תכופות ללקוח.

- 1. איזו מתודולוגיה תבחרו: Agile/Scrum או Waterfall?
- 2. נמקו את הבחירה באמצעות שני מאפיינים של הפרויקט.

שאלה 4 - ארכיטקטורה (4 נקודות)

מערכת קיימת בנויה כמונוליט אחד גדול. לאחרונה נוספו משתמשים רבים, וכל שינוי קטן גורם לסיכון גבוה ולבדיקות רבות.

1. הציעו כיוון ארכיטקטוני לשיפור.
2. כתבו יתרון אחד וסיכון אחד של הכיוון שהצעתם.

נספח תשובות מקוצר לבדיקה עצמית

הנספח נועד לתרגול בלבד. במבחן אמיתי ייתכן שהמרצה ידרוש ניסוח או נימוק שונה.

חלק א - אמריקאיות

שאלה	תשובה	הסבר קצר
1	ב	פעולה שהמערכת צריכה לבצע.
2	ב	Actor הוא גורם חיצוני שמתקשר עם המערכת.
3	ב	Generalization הוא יחס "סוג של".
4	ב	Sequence מציג הודעות לאורך זמן.
5	ב	שכבות מפרידות אחריות ומקלות תחזוקה.
6	ב	Agile מתאים לשינויי דרישות ומשוב תכוף.
7	ב	Acceptance Criteria מגדירים הצלחה באופן מדיד.
8	א	State Machine מציג מצבים ומעברים.

חלק ב - נכון / לא נכון

1	2	3	4	5	6	7	8	9
נכון	לא נכון	נכון	נכון	נכון	נכון	לא נכון	נכון	לא נכון

חלק ג ד - כיווני תשובה

- **Use Case**: סטודנט ומזכירות הם שחקנים. "אישור חריגת מכסה" מתאים ל-extend כי הוא קורה רק בתרחיש חריג.
- **Class Diagram**: Librarian יורש מ-User; משתמש יכול להיות קשור ל-0..* השאלות; אפשר להוסיף מחלקת Reservation.
- **Architecture**: API Gateway מרכז ניתוב ואימות אך עלול להפוך לנקודת כשל. DB משותף יוצר תלות חזקה בין שירותים.
- **תכן**: נפגע עיקרון Single Responsibility. אפשר לפצל ל-OrderRepository, PricingService, ValidationService, NotificationService.
- **דרישות**: פונקציונלית - קביעת תור, ביטול תור; לא פונקציונלית - זמן תגובה, זמינות, אבטחה.
- **מתודולוגיה**: Agile/Scrum מתאים כשהדרישות משתנות ונדרש משוב תכוף.